

06 — Frontend State and Data Models

Product: Endpoint IQ **Document:** State and Data Models

These types mirror the backend data model in §5 of the architecture document (projects, roles, credentials, sessions, endpoints, test_runs, test_results, reports), plus an inferred User/AIAnalysis shape where the source implies but doesn't fully enumerate fields.

1. Core Domain Types

User (Platform User)

Open Decision: Not defined in source (see platform-auth Open Decision in 02- and 05-). Shape inferred as minimally viable for a login/session flow.

```
interface User {  
  id: string;  
  email: string;  
  name?: string;  
  createdAt: string; // ISO date  
}
```

Project

Maps directly to the projects table (id, name, swagger_url).

```
interface Project {  
  id: string;  
  name: string;  
  swaggerUrl: string;  
  createdAt: string;  
  updatedAt: string;  
  // Derived/aggregated fields useful for list/overview views (Proposed):  
  endpointCount?: number;  
  latestRunStatus?: TestRunStatus;  
}
```

Endpoint

Maps to the endpoints table (id, project_id, method, path, schemas).

```
type HttpMethod = "GET" | "POST" | "PUT" | "PATCH" | "DELETE";  
  
interface Endpoint {  
  id: string;  
  projectId: string;  
  method: HttpMethod;  
  path: string;  
  description?: string;  
  requestSchema?: JsonSchema; // from imported OpenAPI spec  
  responseSchema?: JsonSchema; // from imported OpenAPI spec  
}
```

```
// Generic placeholder for OpenAPI JSON Schema fragments
type JsonSchema = Record<string, unknown>;
```

Role

Maps to the roles table (id, project_id, name).

```
interface Role {
  id: string;
  projectId: string;
  name: string; // e.g. "Admin", "Client", "Company"
}
```

Credential

Maps to the credentials table (id, role_id, username, password [encrypted]). **The frontend type never carries a plaintext password after submission** — it is write-only from the client's perspective.

```
// What the frontend SENDS (write-only, never stored/echoed):
interface CredentialInput {
  roleId: string;
  username: string;
  password: string;
}

// What the frontend RECEIVES back (no password field, ever):
interface Credential {
  id: string;
  roleId: string;
  username: string;
  hasPassword: boolean; // whether a credential has been set, without exposing it
  updatedAt: string;
}
```

Session (Backend-Internal — Not Directly Consumed by Frontend)

Maps to the sessions table (id, credential_id, access_token, expires_at). Per the architecture's explicit security model (§4.3, §6), **the frontend never reads or stores this data** — tokens live in Redis and HttpOnly cookies. Included here only for completeness/traceability to the backend data model.

```
// Documented for traceability only — NOT used in any frontend service or store.
interface Session {
  id: string;
  credentialId: string;
  // accessToken intentionally omitted — never exposed to the frontend
  expiresAt: string;
}
```

Test Run

Maps to the test_runs table (id, project_id, start_time, status).

```

type TestRunStatus = "pending" | "running" | "completed" | "failed";

interface TestRun {
  id: string;
  projectId: string;
  startTime: string;
  endTime?: string;
  status: TestRunStatus;
  // Derived progress fields (Proposed – see 05-Frontend-API-Integration.md §16):
  jobsCompleted?: number;
  jobsTotal?: number;
  currentEndpoint?: string;
}

```

Test Result

Maps to the test_results table (id, run_id, endpoint_id, status_code, response).

```

type TestResultOutcome = "passed" | "failed" | "warning";

interface TestResult {
  id: string;
  runId: string;
  endpointId: string;
  roleId: string;
  outcome: TestResultOutcome;
  statusCode: number;
  expectedStatusCode?: number;
  responseTimeMs: number;
  requestHeaders: Record<string, string>;
  requestBody?: unknown;
  responseHeaders: Record<string, string>;
  responseBody?: unknown;
  errorMessage?: string;
  severity?: "low" | "medium" | "high" | "critical"; // used for failed/security results
}

```

AI Analysis

Not enumerated as a distinct table in §5, but is explicitly produced by the AI Analysis Engine (§4.8) and stored as part of the analysis pipeline; reports.summary_text implies analysis output is persisted. Shape below is derived from the described output categories (“missing test coverage, anomalous responses, security concerns,” Plan→Act→Observe→Reflect, JSON-schema-validated).

Open Decision: No explicit ai_analysis table is defined in §5’s Data Model table. Recommend backend confirm whether AI analysis is stored per-result, per-run, or both. Frontend type below is **Proposed**, modeled on the described LLM output categories.

```

interface AIAnalysis {
  id: string;
  testResultId?: string; // per-failure analysis
  runId: string;          // run-level analysis (missing coverage, security summary)
}

```

```

    rootCause?: string;
    recommendedFix?: string;
    developerPrompt?: string;           // copyable prompt for the developer to paste into their
    missingCoverage?: string[];
    securityFindings?: SecurityFinding[];
    model: string;                      // which LLM produced this (e.g. "deepseek-r1-local", "claude")
    generatedAt: string;
}

interface SecurityFinding {
    title: string;
    description: string;
    severity: "low" | "medium" | "high" | "critical";
}

```

Report

Maps to the reports table (id, run_id, file_url, summary_text).

```

interface Report {
    id: string;
    runId: string;
    fileUrl: string;
    summaryText: string;
    generatedAt: string;
}

```

2. Enums / Status Values Summary

Type	Values
HttpMethod	GET, POST, PUT, PATCH, DELETE
TestRunStatus	pending, running, completed, failed
TestResultOutcome	passed, failed, warning
Severity (results & security findings)	low, medium, high, critical

3. Relationships

Directly reflecting §5's entity relationship summary:

```

erDiagram
    PROJECT ||--o{ ENDPOINT : has
    PROJECT ||--o{ ROLE : has
    ROLE ||--o{ CREDENTIAL : holds
    CREDENTIAL ||--o{ SESSION : produces
    PROJECT ||--o{ TEST_RUN : schedules
    TEST_RUN ||--o{ TEST_RESULT : produces
    ENDPOINT ||--o{ TEST_RESULT : "covered by"
    TEST_RUN ||--|| REPORT : outputs
    TEST_RESULT ||--o{ AI_ANALYSIS : "analyzed by"

```

4. Form Types

Form state types live in each feature's `types.ts`, separate from the domain types above (domain types represent server truth; form types represent in-progress edits and carry their own validation state).

```
// features/roles/types.ts
interface RoleFormState {
  name: string;
  errors: Partial<Record<"name", string>>;
}

// features/spec-import/types.ts
interface ImportFormState {
  projectName: string;
  swaggerUrl: string;
  errors: Partial<Record<"projectName" | "swaggerUrl", string>>;
  status: "idle" | "submitting" | "success" | "error";
}
```

5. UI State Types

```
// Shared, generic – components/ or stores/
interface ToastMessage {
  id: string;
  type: "success" | "error" | "info" | "warning";
  message: string;
}

interface TableFilterState {
  status?: TestResultOutcome[];
  method?: HttpMethod[];
  sortBy?: string;
  sortDir?: "asc" | "desc";
  page?: number;
  pageSize?: number;
}
```

6. State Location Summary

Data	Location	Rationale
User (current session)	Global store (stores/authStore)	Needed app-wide, rarely changes
Project, Endpoint, Role, Credential (metadata), TestRun, TestResult, AIAnalysis, Report	Server-state cache (React Query or equivalent), keyed by resource ID	Always re-fetchable from backend; avoids stale duplication
Live TestRun progress	Server-state cache updated via WS/polling hook	Still “server state,” just streamed
Form drafts (RoleFormState, ImportFormState, etc.)	Local component state	Ephemeral, discarded on submit/cancel

Data	Location	Rationale
Table filters/sort/pagination	Local component state (URL-synced recommended)	Ephemeral but should survive refresh via query params
Toasts	Global store (stores/toastStore)	Triggered from anywhere, rendered at app shell level
Modal open/closed	Local component state	Scoped to the component that owns the modal

Cross-references: 05-Frontend-API-Integration.md (request/response usage of these types), 07-Frontend-Test-Results-and-AI-Report-UI.md (heaviest consumer of TestResult/AIAnalysis).